BugBounty-AI — Technical Documentation

Automated AI-Assisted Reconnaissance & Asset Discovery Engine

This document provides a comprehensive technical overview of the **BugBounty-AI** project located at **`D:/projects/BugBoumty`**. It covers the system architecture, background worker queues, recon scanner pipeline, and integration endpoints.

---

1. Executive Summary

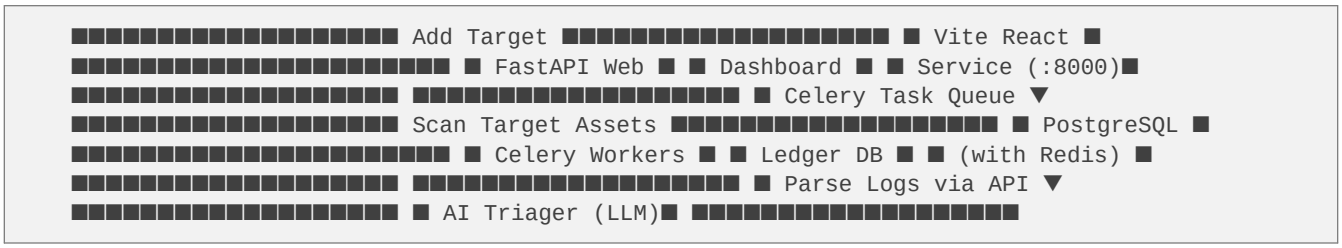
BugBounty-AI is a robust, distributed reconnaissance scanner built to automate defensive threat triage and attack surface mapping. It handles subdomain discovery, active port scanning, and web path crawling concurrently. Discovered assets are stored in a relational database ledger (PostgreSQL) and parsed by AI endpoints (OpenAI GPT or local Ollama instances) to highlight high-priority exploit paths.

---

2. System Architecture

The application is structured for distributed scalability:

- **`api/`**: FastAPI backend exposing REST endpoints for task triggers, targets, and result queries.
- **`workers/`**: Celery worker nodes tasked with running CLI recon utilities in parallel.
- **`db/`**: PostgreSQL database store mapping networks, ports, and live domains.
- **`ai/`**: Parsing adapters interfacing with OpenAI APIs or Ollama instances to evaluate logs.



A. Distributed Recon Pipeline

The platform leverages standard offensive CLI utilities wrapped inside containerized Python execution environments:

1. **`subfinder`**: Discovers subdomains for the target origin via passive DNS queries.
2. **`httpx`**: Probes discovered domains to confirm online HTTP/HTTPS web servers, status codes, and titles.
3. **`katana`**: Crawls active web pages to identify input endpoints, forms, script sources, and API routes.

B. Celery Task Queue & Redis

Scanning heavy domains is a blocking operation. FastAPI handles requests asynchronously by offloading scan execution to a **Celery Worker Pool** backed by a **Redis Broker**. Tasks are executed sequentially or in parallel depending on the scale of the worker pool, avoiding server CPU exhaustion during intensive scans.

---

3. Database Schema Layout

The PostgreSQL ledger stores the state of discovered network assets:

- **`Target`**: Main company domain or IP range defined by user.
- **`Subdomain`**: Subdomains resolved during subfinder scans.
- **`PortScan`**: Ports identified as open, with service name banners.
- **`CrawlRoute`**: Absolute URLs and parameters extracted by the crawling phase.
- **`AI_Flag`**: Custom alerts flagged by the LLM (potential vulnerability markers).

---

4. API Endpoints

The API is exposed via FastAPI on port **`8000`**.

Method	Endpoint	Description
`GET`	**`/`**	Checks api operational status.
`POST`	**`/targets/`**	Adds a company domain to the scanning pool and queues a recon background task.
`GET`	**`/targets/`**	Returns list of all targets and overall scan states.
`GET`	**`/targets/{id}/`**	Detailed overview for a target.
`GET`	**`/targets/{id}/subdomains`**	Returns all subdomains discovered for the given target ID.
`GET`	**`/targets/{id}/ports`**	Returns list of open ports and associated services.

---

5. Configuration & Setup

Create a **`.env`** file in the root directory:

```
# Database Settings POSTGRES_USER=postgres POSTGRES_PASSWORD=dbpassword
POSTGRES_DB=bugbounty_ai DATABASE_URL=postgresql://postgres:dbpassword@db:5432/bugbounty_ai
# Broker Settings REDIS_URL=redis://redis:6379/0 # AI Configuration OPENAI_API_KEY=sk-...
OLLAMA_HOST=http://localhost:11434
```

---

6. How to Run the Application

The entire platform is containerized using Docker and Docker Compose.

Step 1: Deploy Services

1. Navigate to the project root:

```
cd D:/projects/BugBoumty
```

2. Build and launch all containers:

```
docker-compose up --build
```

This starts the API server, Celery worker pool, Redis, and PostgreSQL.

Step 2: Accessing the Systems

- **API Documentation**: http://localhost:8000/docs (Swagger UI)
- **Frontend Client**: http://localhost:5173 (if starting the Vite server locally inside the `frontend/` directory via `npm run dev`)